

note article system

note-article-system

完全マニュアル — ローカルWebアプリ版 (ver4)

たこすけ式 note記事 生成・投稿システム / ブラウザで操作

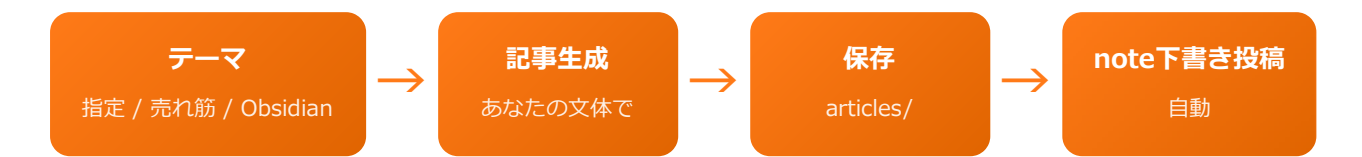
目次

1. このアプリについて
2. 初回セットアップ (はじめに)
3. 起動と画面の見方
4. 記事をつくる (5つの方法)
5. ナレッジ (文体・構成の管理と切替)
6. 右パネル (テーマ・記事の管理)
7. 会話履歴・中断
8. 仕組みとデータ
9. 困ったときは
10. ファイル構成

1. このアプリについて

テーマ決め → 記事生成 → note.com への下書き投稿までを、ブラウザの画面でボタン操作できる、たこすけ式のnote記事自動化ツールです。

ver4 から、これまでのターミナル操作をやめて **ローカルWebアプリ**になりました。中身（記事生成・投稿・テーマ管理）はそのままに、入口が「ブラウザ画面」になっています。



POINT 操作はすべて画面のボタン or チャット欄から。コマンドを打つ必要はありません。

2. 初回セットアップ（はじめに）

必要なもの

項目	説明
Python 3.x	記事投稿・サーバーの実行に使用
Google Chrome	note.com への自動投稿に使用
claude CLI	記事生成の本体。 <code>npm install -g @anthropic-ai/claude-code</code>
Pythonパッケージ	selenium / Pillow / psutil / flask（不足分は自動導入）

かんたんセットアップ

- 1

フォルダ内の `start_webapp.bat` をダブルクリック。ブラウザで `http://127.0.0.1:8788` が自動で開きます。
- 2

画面左上の  **セットアップ（初回設定をガイド）** を押す。環境を自動チェックし、足りないものだけ1ステップずつ案内します。

3

左の  **note.com にログイン** を押す → 専用のChromeウィンドウが開くので、そこで note.com にログイン。完了すると窓は自動で閉じます。

△ ログインは必ず「自動で開いた専用ウィンドウ」で行ってください。普段使いのChromeでログインしても投稿システムには反映されません（保存先プロフィールが別のため）。

POINT 画面上部に赤い「サーバーに接続できません」バナーが出ていなければ、接続は正常です。

3. 起動と画面の見方

`start_webapp.bat` をダブルクリック → ブラウザで `http://127.0.0.1:8788` 。終了するときは、起動時に出る黒いウィンドウ（コマンドプロンプト）を閉じます。



場所	役割
ヘッダー	モデル／モード切替、 筆者・ 構成の参照切替、 履歴、 新規
左パネル	操作メニュー（はじめに／記事をつくる／ナレッジ／フォルダ）
中央	チャット。生成の進捗や報告が表示される。送信／中断／キャンセル
右パネル	環境ステータス／テーマ残数・一覧／記事（未投稿・投稿済）

モードについて

ヘッダーの「モード」は通常 **フルオート**でOK（確認なしで生成～投稿まで実行）。慎重に進めたいときは「制限」「計画のみ」も選べます。

4. 記事をつくる（5つの方法）

① テーマ指定で1件 生成 + 投稿

ボタンを押すとテーマを聞かれます。入力すると、あなたの文体で記事を生成 → 保存 → note に下書き投稿まで一気に実行します。

② 売れ筋からテーマ取得

キーワードを入力すると、**note の非公式API**から売れ筋の有料記事を収集し、それを分析して30個のテーマ案を作って `themes.json` に追加します。

POINT ver4 ではブラウザ自動操作（スクレイピング）をやめ、安全な公開APIに切替。「今売れている記事(24h)」 + 「スキの多い記事」をバランスよく集め、丸パクリにならないよう"切り口の型"を抽出してテーマ化します。

③ 複数件 生成（一覧から選択）

未着手テーマの**残数と全一覧**が表示されます。チェックボックスで作りたいものを選び、「下書き投稿まで／保存のみ」を選んで一括生成。

④ テーマを追加

自分で思いついたテーマを `themes.json` の未着手に追加します。

⑤ Obsidianから生成（フォルダ指定）

Obsidian のメモ（.md）から記事を作ります。「📁 **フォルダを開いて選ぶ**」でフォルダ選択ダイアログから直接指定でき、サブフォルダのドリルダウンも可能。**件数指定**と「**下書き投稿まで／保存のみ**」を選んで一気通貫で実行します。

5. ナレッジ（文体・構成の管理と切替）

「どんな文体・思考で書くか（筆者ルール）」と「どんな構成で書くか（記事構成）」を、**あなたが自由に編集・切替**できます。記事生成時は、選択中（★）のものが参照されます。

筆者ルール / 記事構成

- カテゴリごとに**複数のキャラ／構成を保存**でき、**タブで切替**。★が付いているものが「参照中」
- 編集・新規作成・リネーム・削除が可能
- ヘッダーのドロップダウン（ 筆者 /  構成）でワンタッチ切替もできます

参照情報から自動生成

真似たい文章サンプルや「こういうトーンにしたい」という方針を貼り付けると、**既存ナレッジの構造を踏襲**して新しい筆者ルール・記事構成を自動作成します。「生成後そのまま参照先にする」にチェックすれば即適用。

POINT 複数のキャラ（例：たこすけ／ビジネス女子／毒舌系）を使い分けたいときに便利。ヘッダーで切り替えるだけで、次の記事から文体が変わります。

6. 右パネル（テーマ・記事の管理）

テーマ

- 未着手テーマを**全件表示**。各テーマの**文字をクリック**でそのテーマを**単体生成**
- チェックを入れて  **削除**（全選／解除あり）で不要なテーマを掃除できます

記事

- **未投稿**：上の「 **未投稿 N件** を下書き投稿」でまとめて投稿。各記事はクリックで本文編集
- **投稿済み**：チェック+  でローカルファイルを削除（※note.com 上の下書きは消えません）

POINT 下書き投稿に成功すると、対応するテーマが `themes.json` で「未着手 → 完了」へ自動で移動します。

7. 会話履歴・中断

- 🕒 **履歴**：過去の会話がブラウザに自動保存され、一覧から開いて続きから再開できます。各会話の削除・全削除も可能
- 🔄 **新規**：新しい会話を開始（過去の会話は履歴に残ります）
- ⏸ **中断**：実行中の生成を途中で止める／**キャンセル**：入力欄をクリア

8. 仕組みとデータ

場所 / ファイル	役割
themes.json	テーマ管理（pending=未着手／done=完了）。投稿成功で自動更新
articles/	生成記事（未投稿）。投稿後は articles/posted/ へ移動
prompt_knowledge/筆者/	筆者ルール（複数）。.active が参照中のファイル
prompt_knowledge/手順/	記事構成（複数）。.active が参照中のファイル
note_poster.py	note.com への下書き投稿（Selenium） + themes.json 自動更新
note_scraper.py	売れ筋リサーチ（note 非公式API・Chrome不要）
webapp/	Webアプリ本体（server.py = サーバー、static/index.html = 画面）

POINT 記事を書く前に、システムは選択中（.active）の筆者ルールと記事構成だけを読み込みます。選んでいないキャラは混ざりません。

9. 困ったときは

症状	対処
赤いバナー／ボタンが反応しない	サーバー未起動。start_webapp.bat を起動し、実ブラウザで http://127.0.0.1:8788 を開く（Claude内のプレビューでは動きません）
機能を追加・更新したのに反映されない	画面だけの変更はCtrl+Shift+Rで再読込。サーバー側（server.py）の変更は再起動が必要。start_webapp.bat は古いサーバーを自動終了してから起動します
「取得失敗（HTTP 404）」	古いサーバーが動いたまま。バッチを再実行して最新版で起動

投稿で止まる

note.com 未ログイン。左の「🔑 ログイン」でログイン

フォルダ選択ダイア
ログが見当たらない

サーバーを動かしているPCのデスクトップに出ます（最前面）。最小化やタスクバーを確認

10. ファイル構成

```
note-article-system_ver4/
├─ start_webapp.bat      ← ダブルクリックで起動
├─ webapp/
│   └─ server.py         ← ローカルサーバー（claude CLI を呼ぶ）
│       └─ static/index.html ← 画面
├─ CLAUDE.md             ← システムへの指示書
├─ note_poster.py        ← note下書き投稿（Selenium）+ themes自動更新
├─ note_scraper.py       ← 売れ筋リサーチ（非公式API・Chrome不要）
├─ themes.json           ← テーマ（未着手 / 完了）
├─ config.json           ← 設定（アカウント・Obsidianパス等）
├─ articles/            ← 生成記事
│   └─ posted/          ← 投稿済み
├─ prompt_knowledge/     ← 執筆ナレッジ（Web UIから編集・切替）
│   └─ 筆者/            ← 筆者ルール・複数／.active が参照中）
│       └─ 手順/        ← 記事構成・複数／.active が参照中）
└─ manual/              ← このマニュアル
```

note-article-system ver4 — ローカルWebアプリ版
起動： `start_webapp.bat` → `http://127.0.0.1:8788`